

COMP2300/6300

Computer Organisation & Program Execution

Dr Uwe Zimmer

Dr Charles Martin

Semester 1, 2019



Week 12: Revision!

info

Exam: Search <https://exams.anu.edu.au/timetable/>

Thursday 06/06/2019, 2:00pm.

180 minutes writing, 15 minutes reading. *No materials permitted.*

- Adams, D - Taylor, J: 7-11 Barry Drive First Floor Left Side
- Taylor, J - Zou, S: Manning Clark Hall

Info for students: <http://quicklink.anu.edu.au/2yl4>

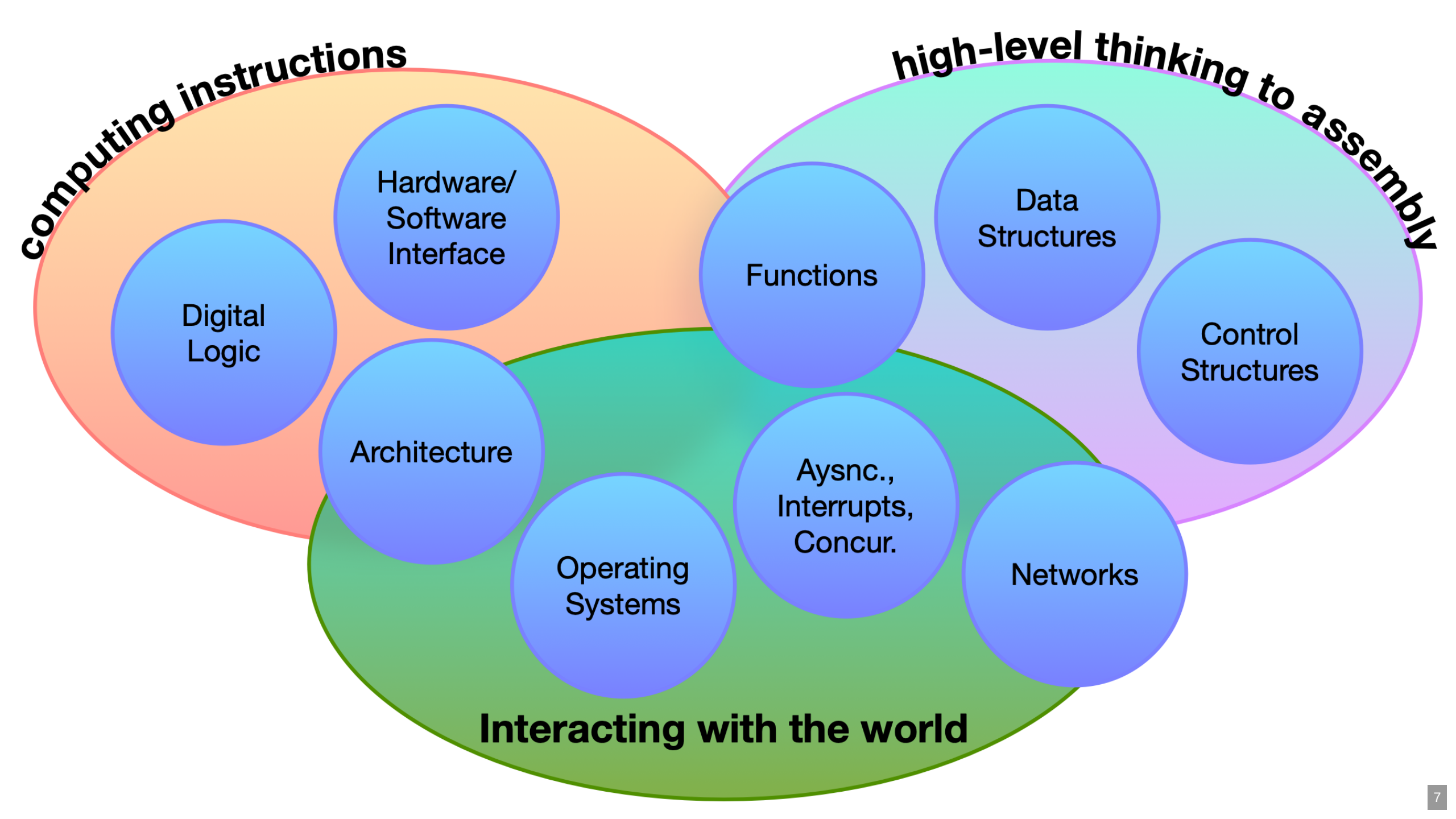
It's been a journey...



Outline

- Tour of the topics
- A few questions from each major topic
- Advice for exams
- Open Questions

What was this course about again?



Digital Logic

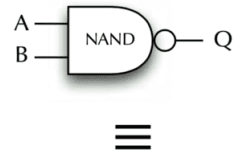
Digital Logic Topics (in short):

- Boolean Algebra
- Combinatorial Logic Functions
- Digital Electronics - Logic Gates
- Binary Encoding, and 2's-Complement
- Adders: Half, Full, Ripple Carry
- Arithmetic Logic Unit
- Simple CPU Architecture

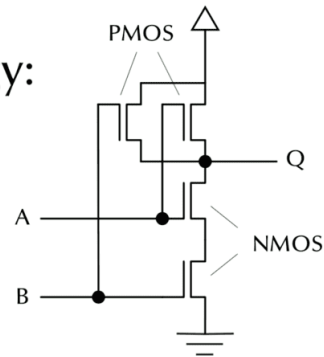
Logic gates

Symbolic: $Q = \overline{A \wedge B}$

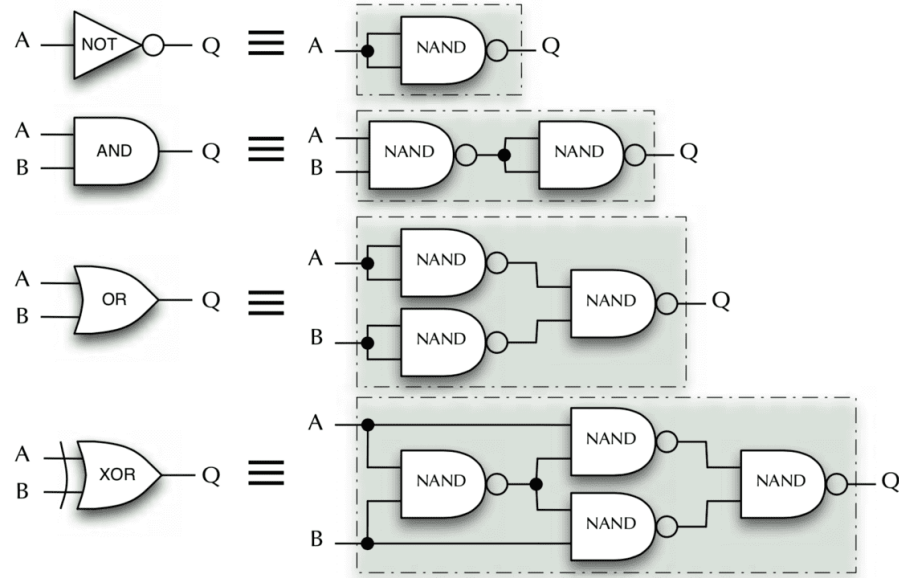
Diagram:



Technology:



Elementary logic gate symbols:



All gates can be represented by NAND or NOR.

talk

1. how many bits can be added together?
2. how long does it take?
3. where does the final carry bit go?

Twos complement representation

The basic idea: define (binary) negative numbers so the adder works.

0	0	1	0	1	0	1	0
+							
?	?	?	?	?	?	?	?
=							
1	0	0	0	0	0	0	0

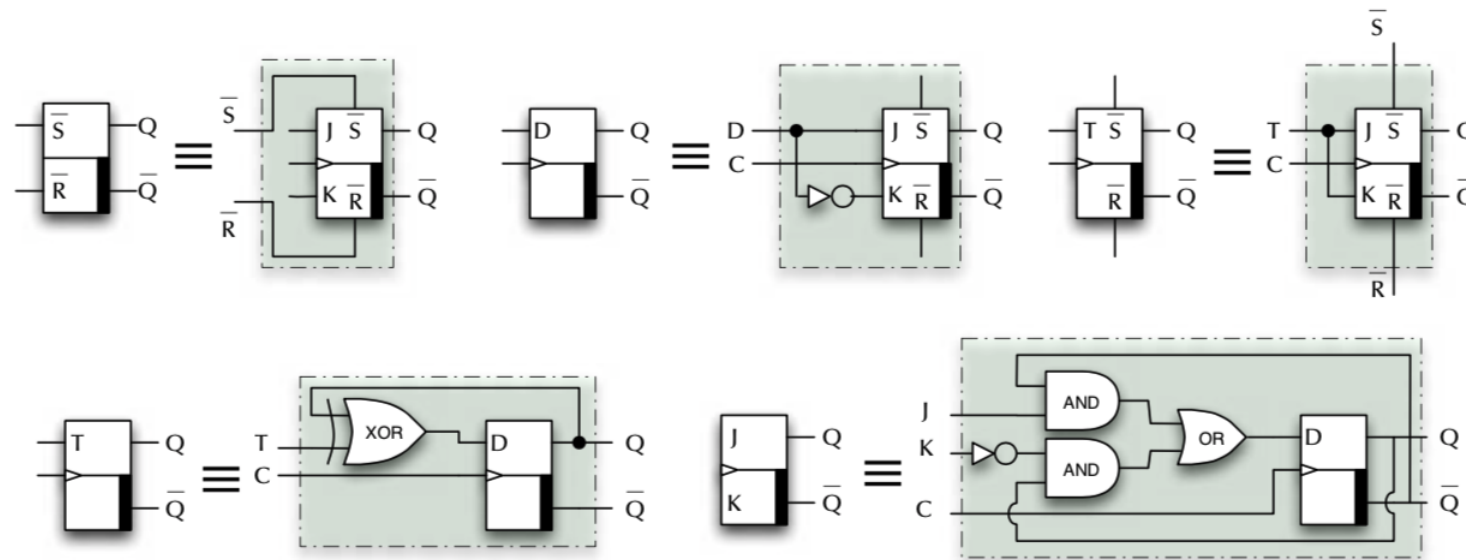
42

-42

256

How do we make a number negative? Invert bits and add one! Why does that work?

Flip-flops



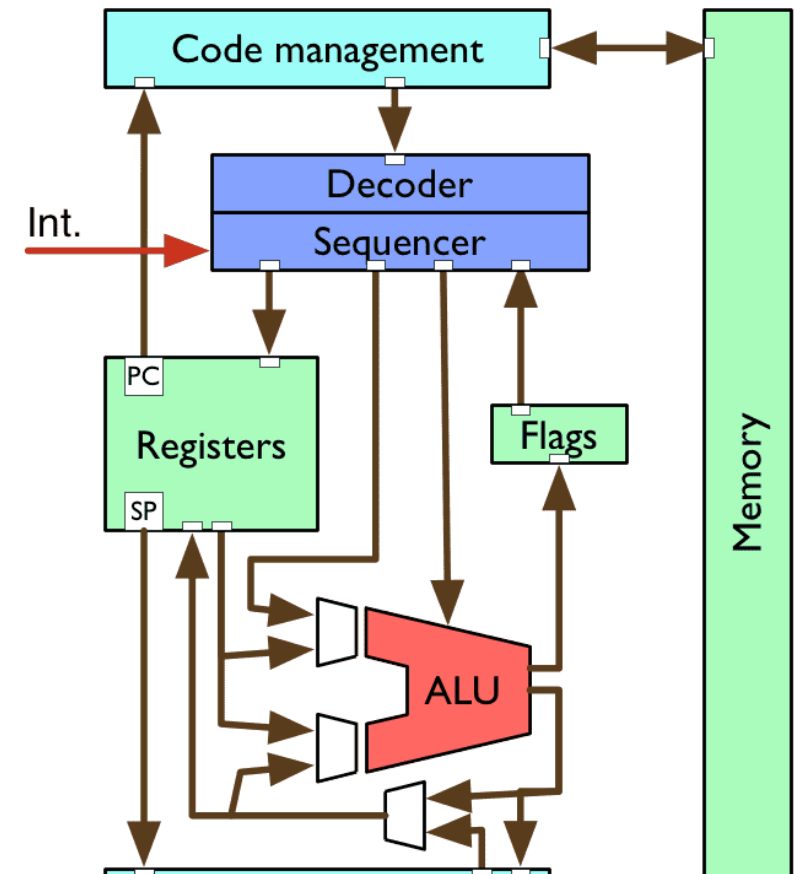
Q_0 Q_1 Q_2 Q_3 Q_4 Q_5 Q_6 Q_7

CPU Architecture

What are the **main components** of a CPU?

Can you **explain** what each of these components do?

We come back to this later...



Hardware/Software Interface

Hardware/Software Interface Topics (in short)

- Structure of an instruction
- Assembly to CPU instructions
- CPU Status Flags (NCZV)
- ARM v7 instructions (adding, subtracting, moving, rotate/shift, bit-wise ops)
- loading and storing from memory
- branch instructions
- Contents of Quick Ref. Card

talk

why do we need both immediate and register versions of some instructions?

NCZV Flags

- **N**egative
- **Z**ero
- **C**arry
- **O**verflow

Negative

This status flag is set when the result of an ALU operation is negative *if interpreted as a two's complement signed integer*

```
movs r0, 5  
movs r1, 6  
subs r2, r0, r1
```

don't forget the `s` suffix

Zero

This status flag is set when the result of an ALU operation is zero

```
movs r5, 5  
movs r6, -5  
adds r4, r5, r6
```

Carry

This status flag is set when the result of an ALU operation requires a “carry out” *if interpreted as an unsigned 32-bit integer* (i.e. it requires 33 or more bits to represent)

```
movs r2, 0xFF000000  
movs r3, 0xFF000000  
adds r5, r2, r3
```

Overflow

This status flag is set when the result of an ALU operation would overflow the min/max value *if interpreted as a twos complement signed integer*

```
movs r0, 0x7FFFFFFF @ largest signed integer  
adds r0, 1
```

```
movs r0, 0x80000000 @ smallest signed integer  
subs r0, 1
```

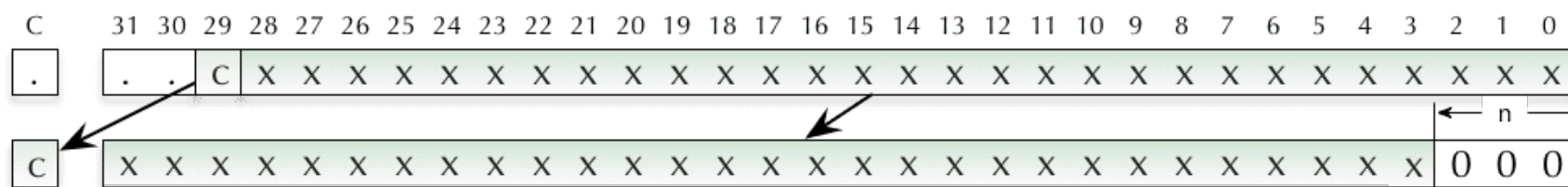
talk

```
movs r0, 5  
movs r1, 6  
subs r2, r0, r1
```

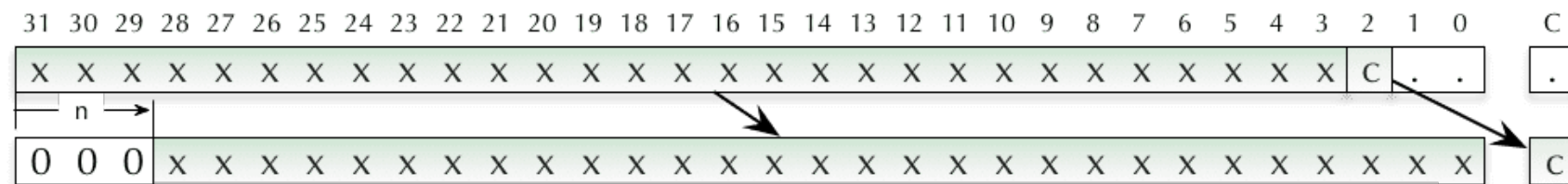
What flags will be set after the `subs` instruction is executed?

		address range
Vendor-specific memory	511 MB	0xFFFFFFFF
		0xE0100000
Private peripheral bus	1 MB	0xE00FFFFF
		0xE0000000
		0xDFFFFFFF
External device	1 GB	
		0xA0000000
		0x9FFFFFFF
External RAM	1 GB	
		0x60000000
		0x5FFFFFFF
Peripherals	0.5 GB	
		0x40000000
		0x3FFFFFFF
SRAM	0.5 GB	
		0x20000000
		0x1FFFFFFF
Code	0.5 GB	
		0x00000000

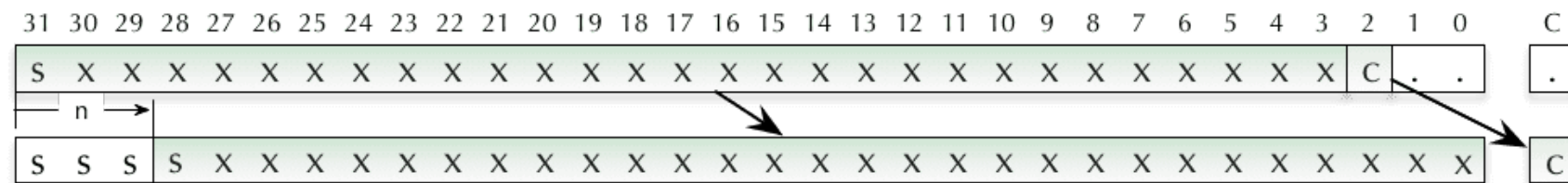
lsl



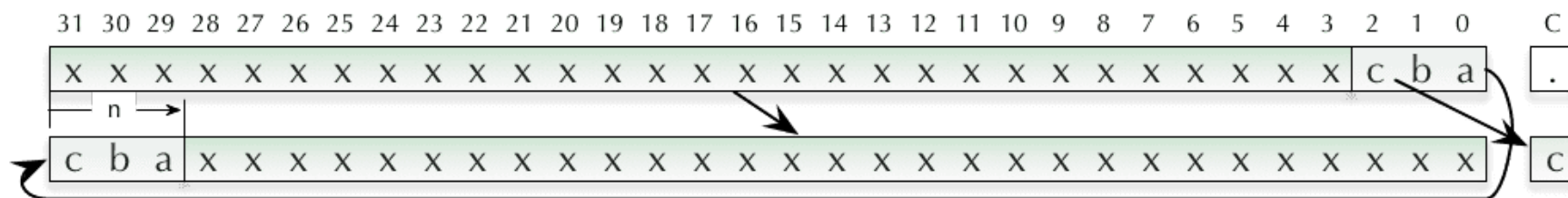
lsr



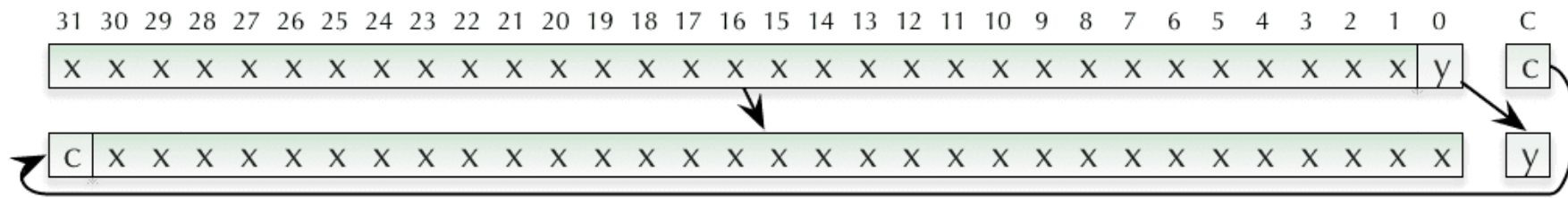
asr



ror



rrx



Loading and Storing

Load:

```
mov r1, 0x200000000 @ put the address in r1
ldr r0, [r1]         @ load the data into r0
```

Store:

```
mov r0, 42
mov r1, 0x200000000
str r0, [r1]
```

Extra Operations

Load less than 32 bits

```
ldrb @ load byte from register  
ldrh @ load halfword from register  
strb @ store byte to register  
strh @ store halfword to register
```

Negative Stack

```
stmdb <Rs>!, {Rgstrs} @ store multiple decrement before  
ldmia <Rs>!, {Rgstrs} @ load multiple increment after  
push {Rgstrs}  
pop {Rgstrs}
```

Conditional branch examples

```
beq <label> @ branch if Z = 1
bne <label> @ branch if Z = 0

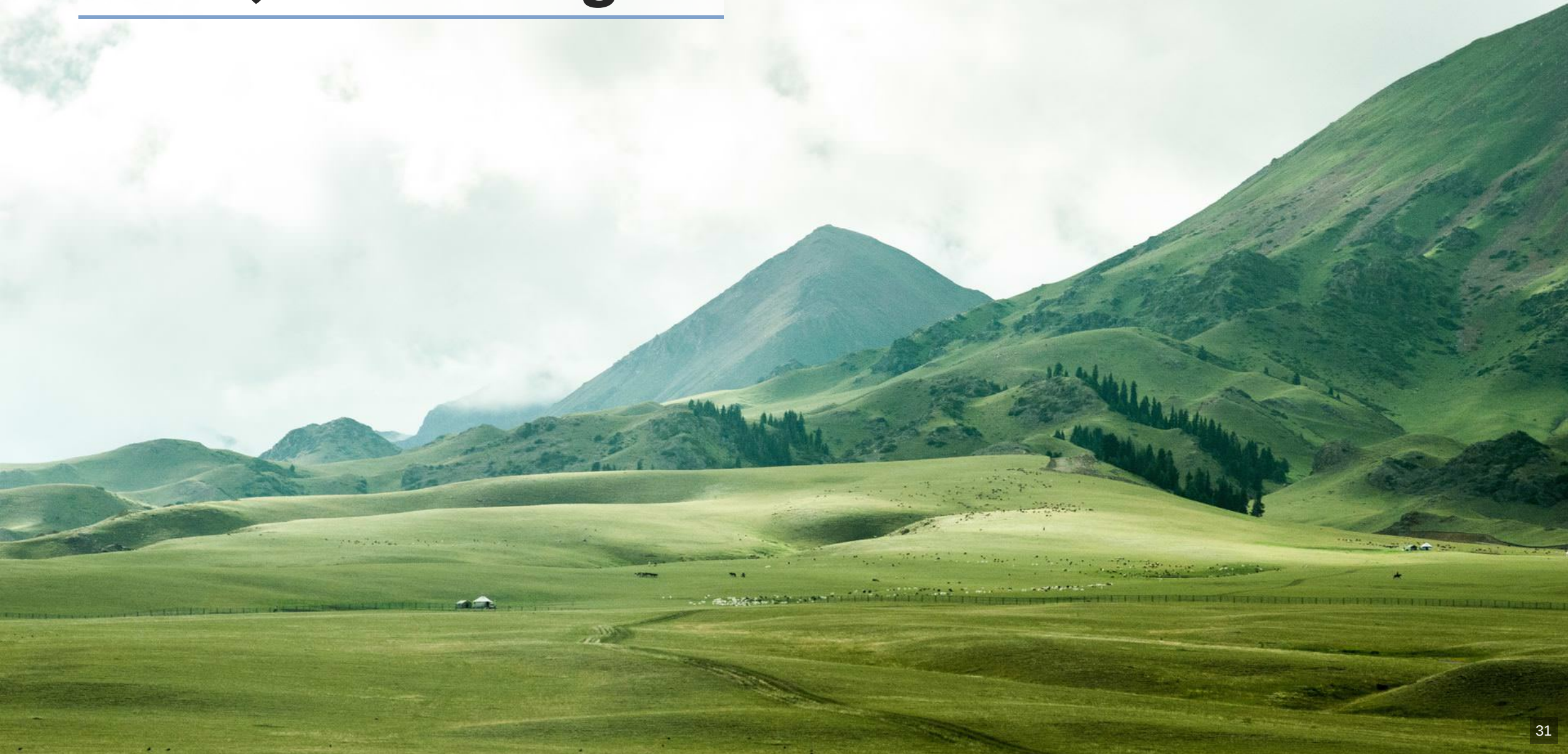
bcs <label> @ branch if C = 0
bcc <label> @ branch if C = 1
bmi <label> @ branch if N = 1
bpl <label> @ branch if N = 0
bvs <label> @ branch if V = 1
bvc <label> @ branch if V = 0
```

Functions

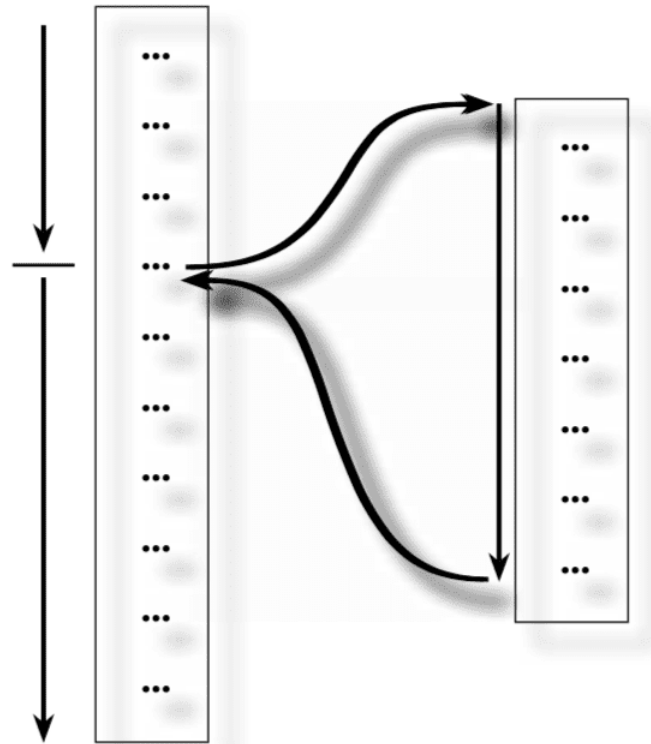
Functions Main Topics (in short)

- There and back again, `bl`, `bx`, and `lr`
- The stack
- Calling conventions
- Functions calling functions
- Functions calling themselves! (a.k.a. recursive functions)
- Local variables, and the stack frame (incl. `sp` and `fp`)
- Relative addressing
- Passing values: by copy and by reference

there, and back again



A function call



AAPCS

The [ARMv7 Architecture Procedure Call Standard](#) is the convention we'll (try to) adhere to in programming our discoboards

The full standard is quite detailed, but the general summary is:

- `r0-r3` are the parameter and scratch registers
- `r0-r1` are also the result registers
- `r4-r11` are callee-save registers
- `r12-r15` are special registers (`ip`, `sp`, `lr`, `pc`)

Store and Load to the stack

@ Push the value in r2 onto the stack

```
str r2, [sp, -4]
```

```
sub sp, sp, 4
```

@ Different one-liners for Push

```
str r2, [sp, -4]!
```

```
push {r2}
```

```
stmdb sp!, {r2}
```

@ Pop the value from the "top" of the stack into r3

```
ldr r3, [sp]
```

```
add sp, sp, 4
```

@ One-liners for Pop

```
ldr r3, [sp], 4
```

```
pop {r3}
```

```
ldmia sp!, {r2}
```

Push and Pop; illustrated

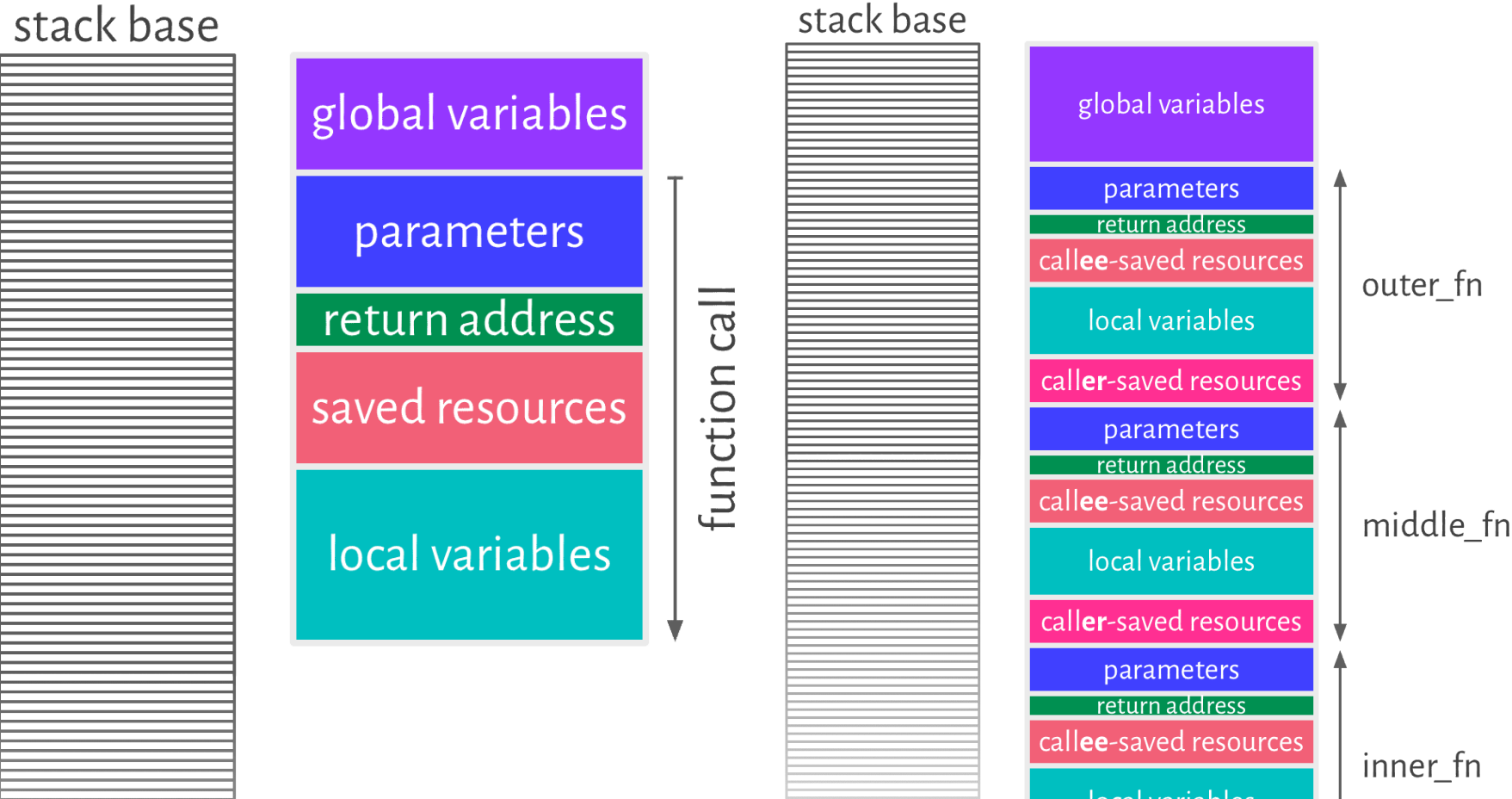
<— Push



Passing by Copy or Reference

		Access	
		by copy	by reference
Information flow	in	by value Parameter becomes a constant inside the function or is copied into a local variable.	by reference (immutable) No write access is allowed while the function runs (also from outside the function).
	out	by result Calling function expects the parameter value to appear in a specific space at return.	by reference (mutable, no read) No read access from inside the function, write access on return.
	in & out	by value result Parameter is copied to a local variable and copied back at return.	by reference (mutable) Function can read and write at any time. Outside code shall not write.

Function stack frame



Recursive Functions: Factorial

```
fact: @ assume input is in r1
    push {lr}
    cmp r1, #1

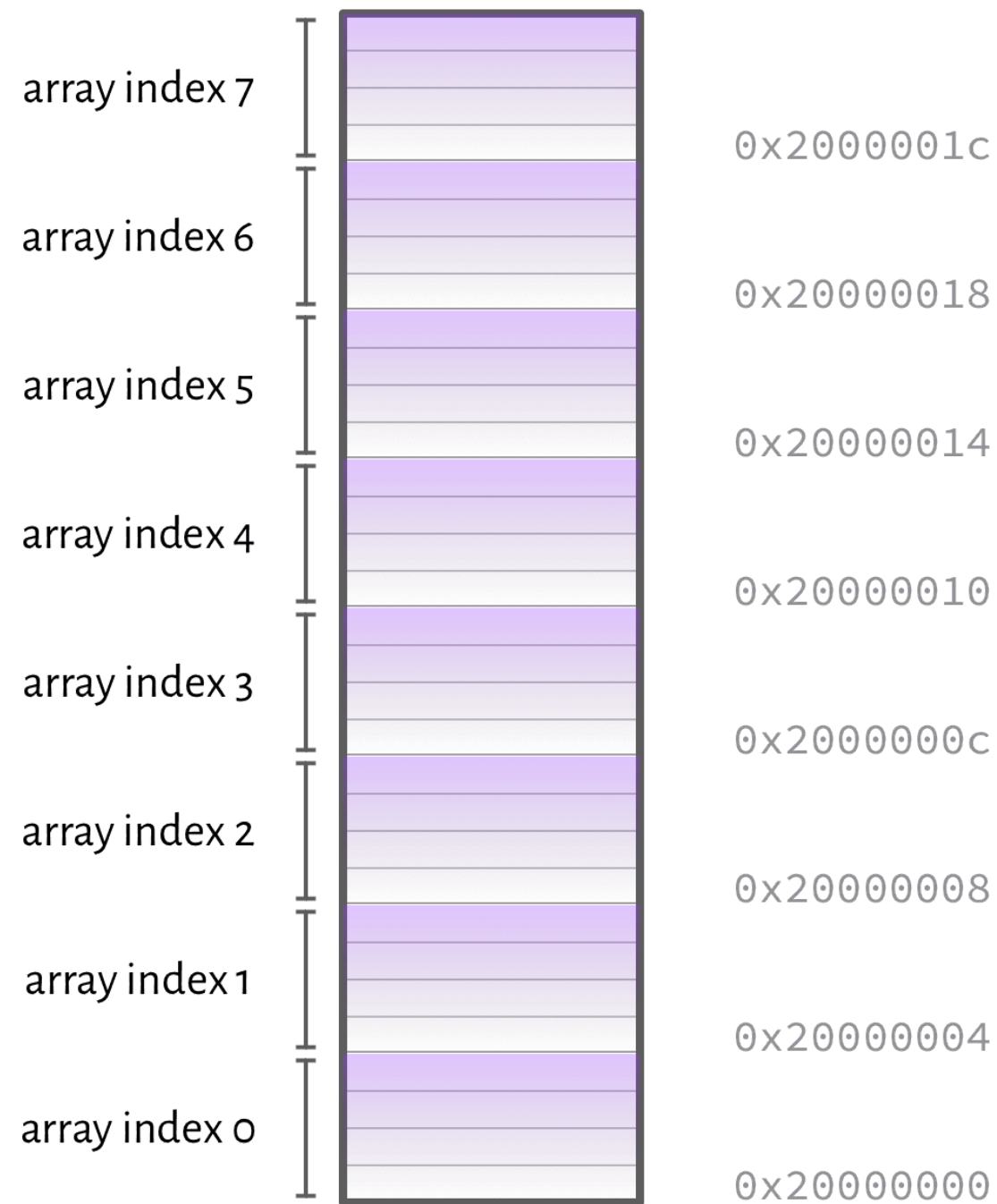
    beq base_case
    @ recursive case
    push {r1}
    sub r1, #1
    bl fact @ get fact(n-1)
    pop {r1}
    mul r0, r0, r1 @ calc fact(n-1) * n
    b continue_code

base_case:
    mov r0, #1
continue_code:
    pop {lr}
    bx lr
```

Data Structures

Data Structures Main Topics (in short)

- **Arrays**
- Structure
- Alignment
- Addressing
- Iterators
- Copying



talk

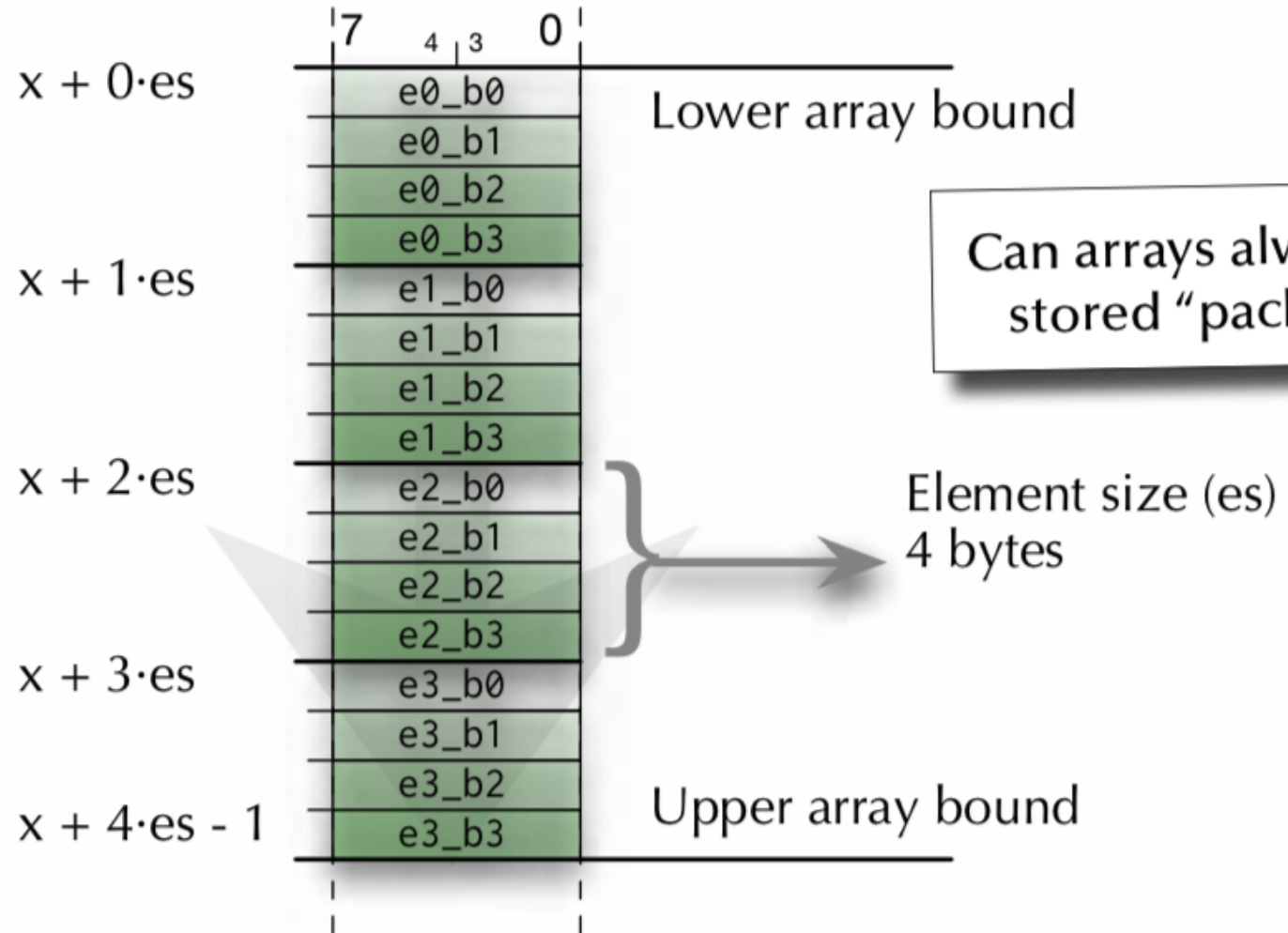
How do we know how big an array is in memory?

Is it possible to write outside the bounds of the array?

Can you make an array where the size can be changed? (mutable array?)

How do we address a particular element in an array?

Array addressing



$x + i \cdot es$

Maybe be good to have $es = 2^n$?

What happens if array bounds are violated?

☞ And who is checking that?

Can arrays always be stored "packed"?

Element size (es)
4 bytes

Upper array bound

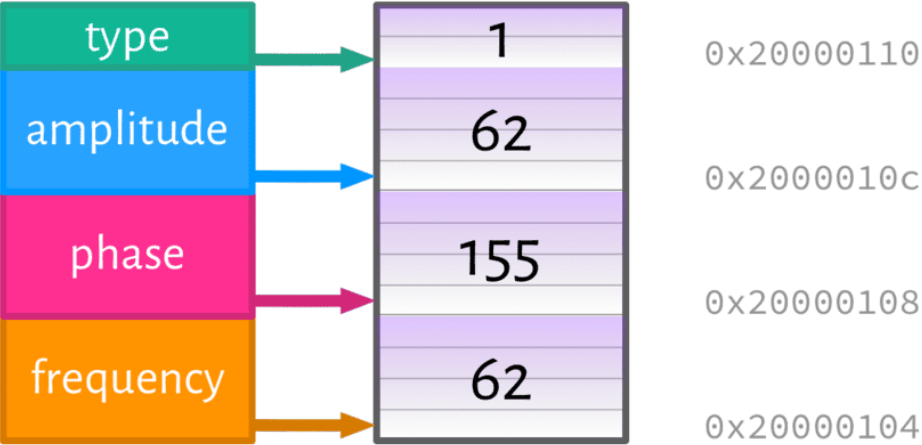
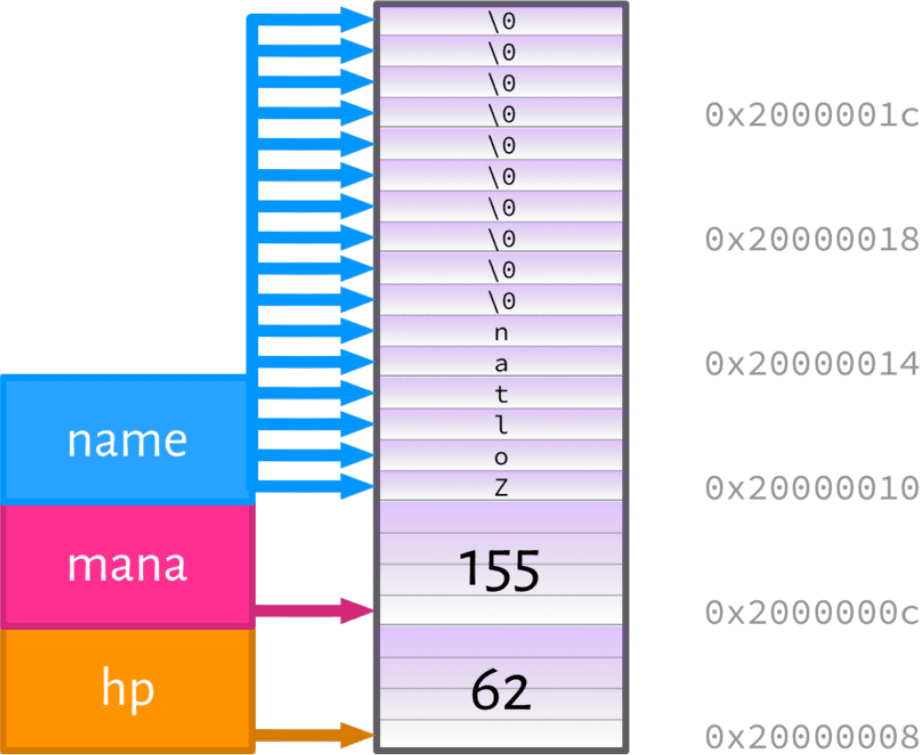
Add up the numbers in an array

```
ldr r0, =array @ base address
mov r1, 0 @ from_index
mov r2, 7 @ to_index

@ setup
mov r3, 0 @ "accumulator" register
mov r4, 4 @ element size

array_sum:
mul r5, r1, r4 @ calculate offset
ldr r6, [r0, r5] @ load from offset
add r3, r6 @ update accumulator
add r1, 1 @ increment index
cmp r1, r2 @ keep looping?
ble array_sum
```

Records



talk

What's the difference between an **array** and a **record**?

Control Structures

Control Structures Main Topics (in short)

- Conditional branching
- Control Structures in Machine Code:
 - if
 - while
 - for

CPSR table

<c>	meaning	flags
eq	equal	Z=1
ne	not equal	Z=0
cs	carry set	C=1
cc	carry clear	C=0
mi	minus/negative	N=1
pl	plus/positive	N=0
vs	overflow set	V=1
vc	overflow clear	V=0
hi	unsigned higher	C=1 $\wedge$ Z=0
ls	unsigned lower or same	C=0 $\vee$ Z=1
ge	signed greater or equal	N=V
lt	signed less	N $\neq$ V
gt	signed greater	Z=0 $\wedge$ N=V

while loop components

```
while Register_1 < 100 loop
  Register_1 := Register_1 ** 2;
end loop;
```

```
while (register1 < 100) {
  register1 = register1 * register1;
}
```

```
while register1 < 100:
  register1 = register1 ** 2
```

```
while Register_1 < 100 do
  Register_1 := Register_1 ** 2;
```

```
while Register_1 < 100 do
  Register_1 = Register_1 ** 2
enddo
```

```
while (Register_1 < 100) {
  Register_1 = Register_1 ** 2;
}
```

1. an expression (if)
2. a boolean condition (if)
3. code inside the loop

control structures gallery - practice these!

```
if "cmp r1, r2", eq, "mov r3, #1", "mov r3, #0"
```

```
for r1, 1, 100 "add r3, r1"
```

```
mov    r1, #1
for:
  cmp   r1, #100
  bgt   end_for
  add   r3, r1
  add   r1, #1
  b     for
end_for:
```

```
cmp    r1, r2
beq     then
mov     r3, #0
b       end_if
then:
  mov    r3, #1
end_if:
```

```
while "cmp r1, #100", lt, "mul r1, r1"
```

```
b       while_condition
while:
  mul    r1, r1
while_condition:
  cmp    r1, #100
  blt    while
```

talk

Which control structures were useful for your assignments?

Is there anything you can do in assembly that goes beyond “typical control structures”?

Asynchronism, Interrupts, and Concurrency

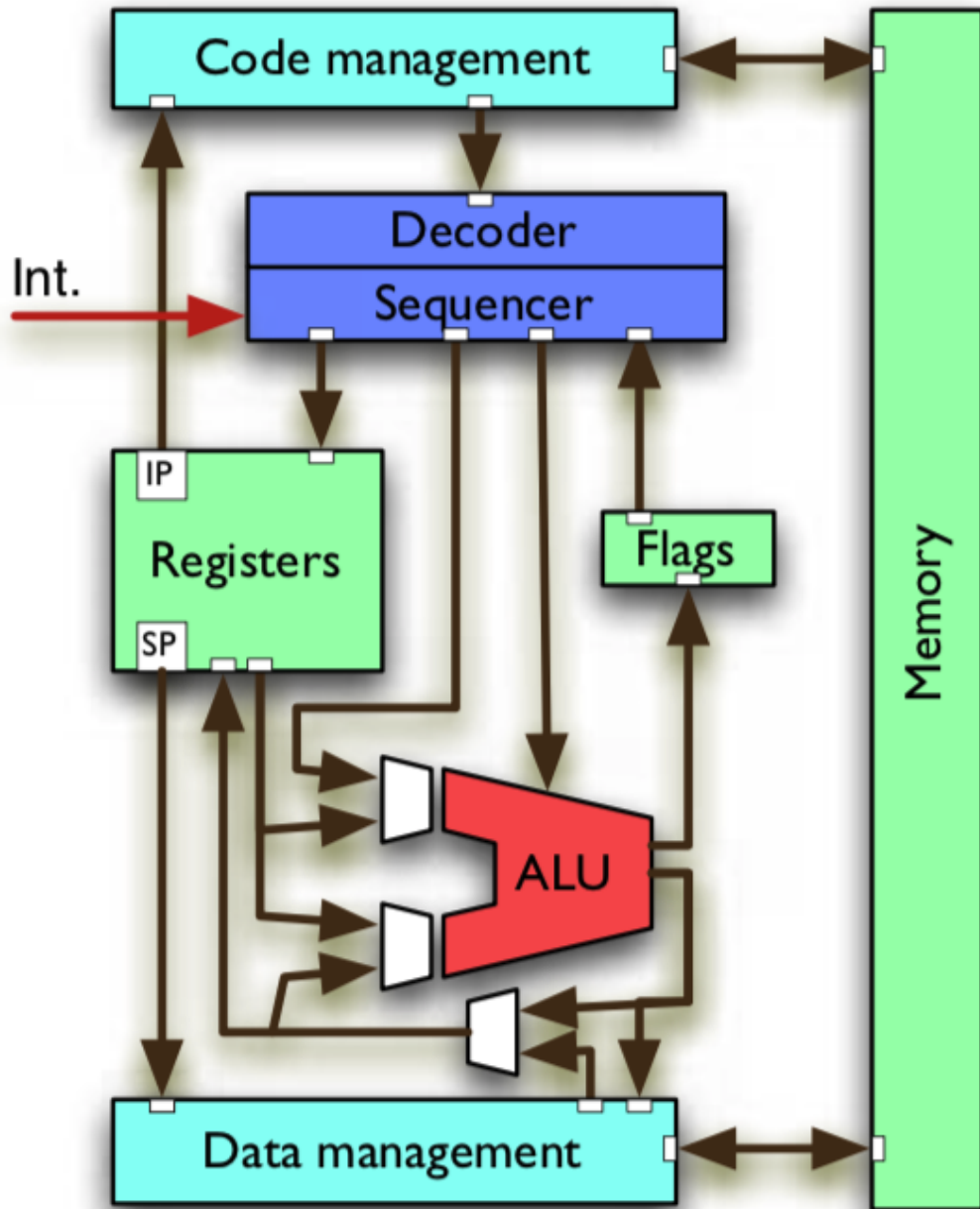
Async Main Topics (in short)

- Interrupts & Exceptions: When and Why?
- What happens during an interrupt?
- How is this related to **parallel computing**?
- Concurrency and Synchronisation
- Race Conditions
- Mutual Exclusion
- Synchronisation
- Passing Data

talk

What's an interrupt? Why are they necessary?

Interrupts

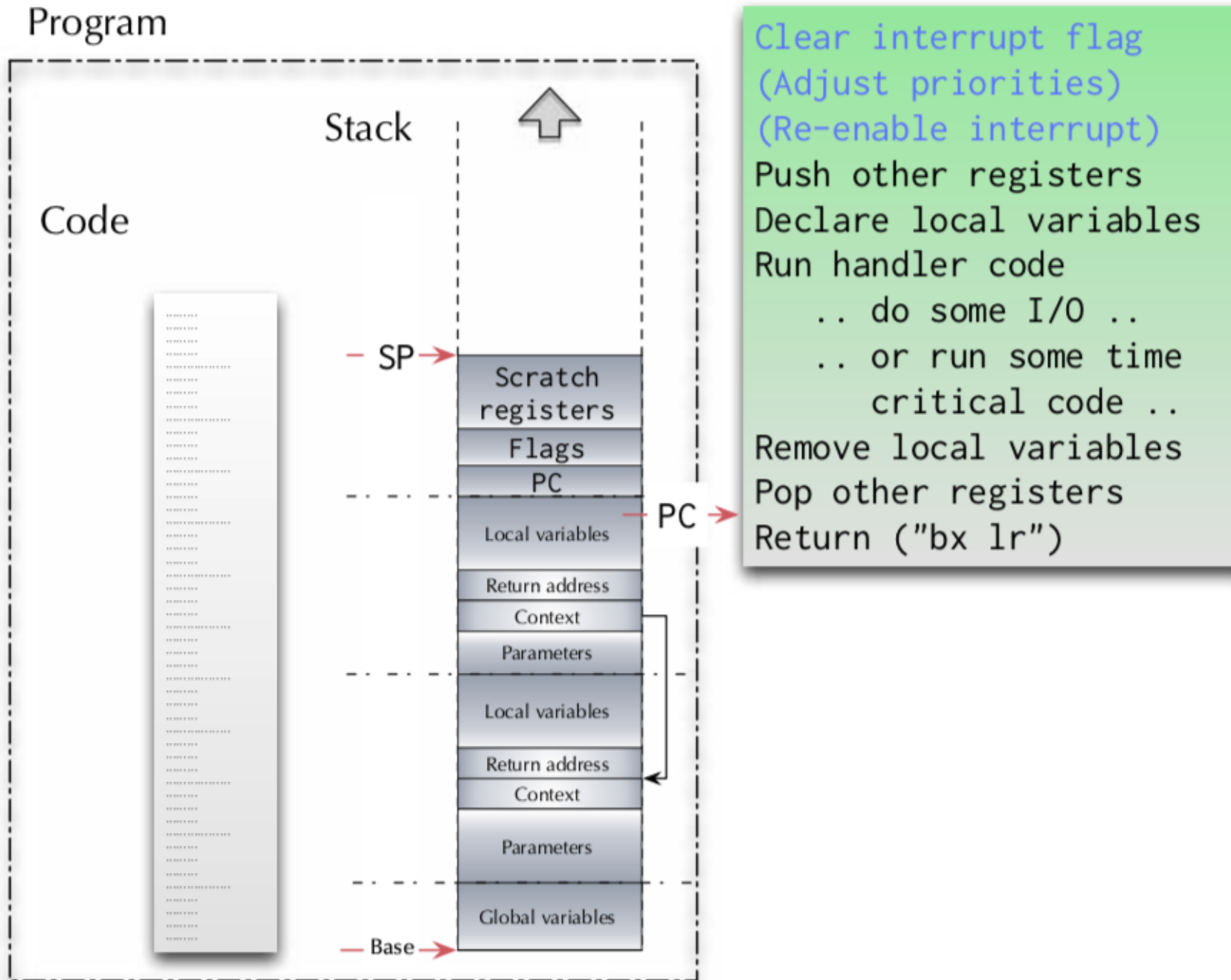


- One or multiple lines wired directly into the sequencer
- ☞ *Required for:*
Pre-emptive scheduling, Timer driven actions, Transient hardware interactions, ...
- ☞ Usually preceded by an external logic ("interrupt controller") which accumulates and encodes all external requests.

On interrupt (if unmasked):

- CPU stops normal sequencer flow.
- Lookup of interrupt handler's address
- Current IP and state pushed onto stack.
- IP set to interrupt handler.

Interrupt handler



Race Conditions and Mutual Exclusion

When the sequence or timing of *threads of execution* has an effect on the outcome.

Can result in bugs! (e.g., in Assignment 3!) What is the value at **Count** in this code?

Mutual exclusion ... or the lack thereof

Count: `.word` 0x00000000

```
ldr    r4, =Count
mov    r1, #1
for_enter:
cmp    r1, #100
bgt    end_for_enter
ldr    r2, [r4]
add    r2, #1
str    r2, [r4]
add    r1, #1
b      for_enter
end_for_enter:
```

```
ldr    r4, =Count
mov    r1, #1
for_leave:
cmp    r1, #100
bgt    end_for_leave
ldr    r2, [r4]
sub    r2, #1
str    r2, [r4]
add    r1, #1
b      for_leave
end_for_leave:
```

Count: **.word** 0x00000000

```
ldr    r4, =Count
mov    r1, #1
```

for_enter:

```
cmp    r1, #100
bgt    end_for_enter
```

```
ldr    r4, =Count
mov    r1, #1
```

for_leave:

```
cmp    r1, #100
bgt    end_for_leave
```

Negotiate who goes first

```
ldr    r2, [r4]
add    r2, #1
str    r2, [r4]
```

Critical section

```
ldr    r2, [r4]
sub    r2, #1
str    r2, [r4]
```

Critical section

Indicate critical section completed

```
add    r1, #1
b      for_enter
end_for_enter:
```

```
add    r1, #1
b      for_leave
end_for_leave:
```

Semaphores

Basic definition (Dijkstra 1968)

Assuming the following three conditions on a shared memory cell between processes:

- a set of processes agree on a variable **S** operating as a flag to indicate synchronization conditions
- an atomic operation **P** on **S** — for ‘passeren’ (Dutch for ‘pass’):
P(S): [as soon as $S > 0$ then $S := S - 1$]  this is a potentially delaying operation
- an atomic operation **V** on **S** — for ‘vrygeven’ (Dutch for ‘to release’):
V(S): [$S := S + 1$]

 then the variable **S** is called a **Semaphore**.

talk

What's a race condition? What can we do about it?

talk

What's mutual exclusion?

Can this be achieved on a discoboard?

How would you do it?

Networks

Networks Main Topics (in short)

- Transmission mediums
- Communications protocols
- Packet switched/circuit switched
- Simplex/duplex
- Topology
- Parallel/Serial
- Timing and Synchronisation
- OSI reference model (7-layers!)

talk

How many transmission mediums can you name?

If you were stuck on a desert island what transmission media could you use to send a message for help?

talk

What is a communications protocol?

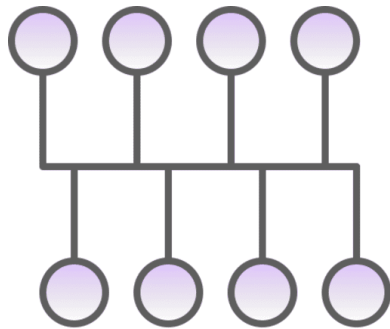
Why would it be needed?

Explain your answer.

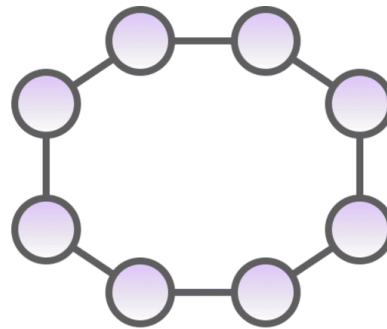
Topology

topology is the way that the nodes are connected to one another (both physically and logically)

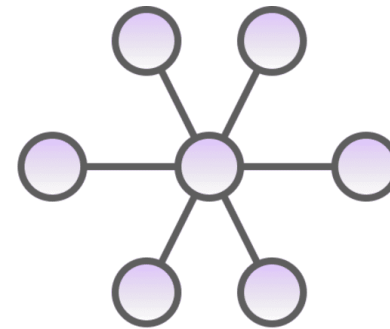
there are several different ways to connect the nodes together, each with pros and cons



bus



ring



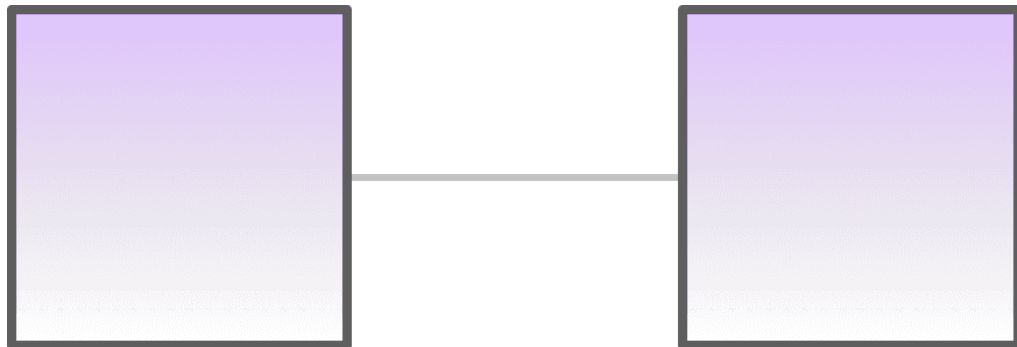
star

Serial vs parallel

serial

data is sent one-bit-at-a-time

fewer bits sent per signal, but simpler



parallel

multiple bits sent simultaneously (e.g. multiple wires)

need to keep all the connections in sync



Synchronous vs Asynchronous

synchronous

transitions on a *clock line*

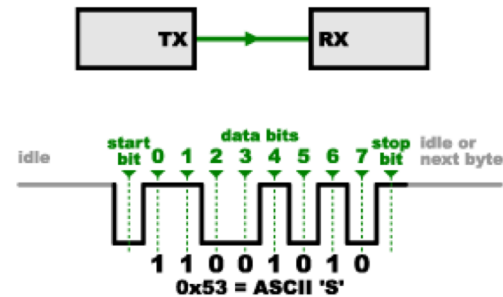
no clock skew issues, but
requires an extra connection

asynchronous

(independent) timers at each end

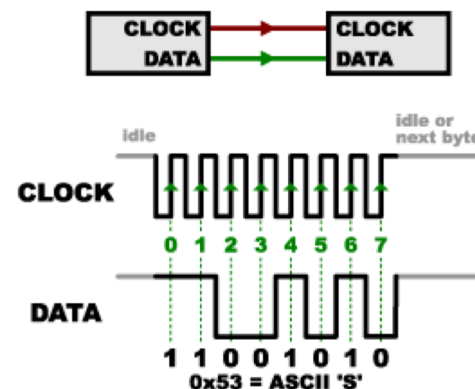
no extra connections required, but more
vulnerable to synchronisation issues

Asynchronous:

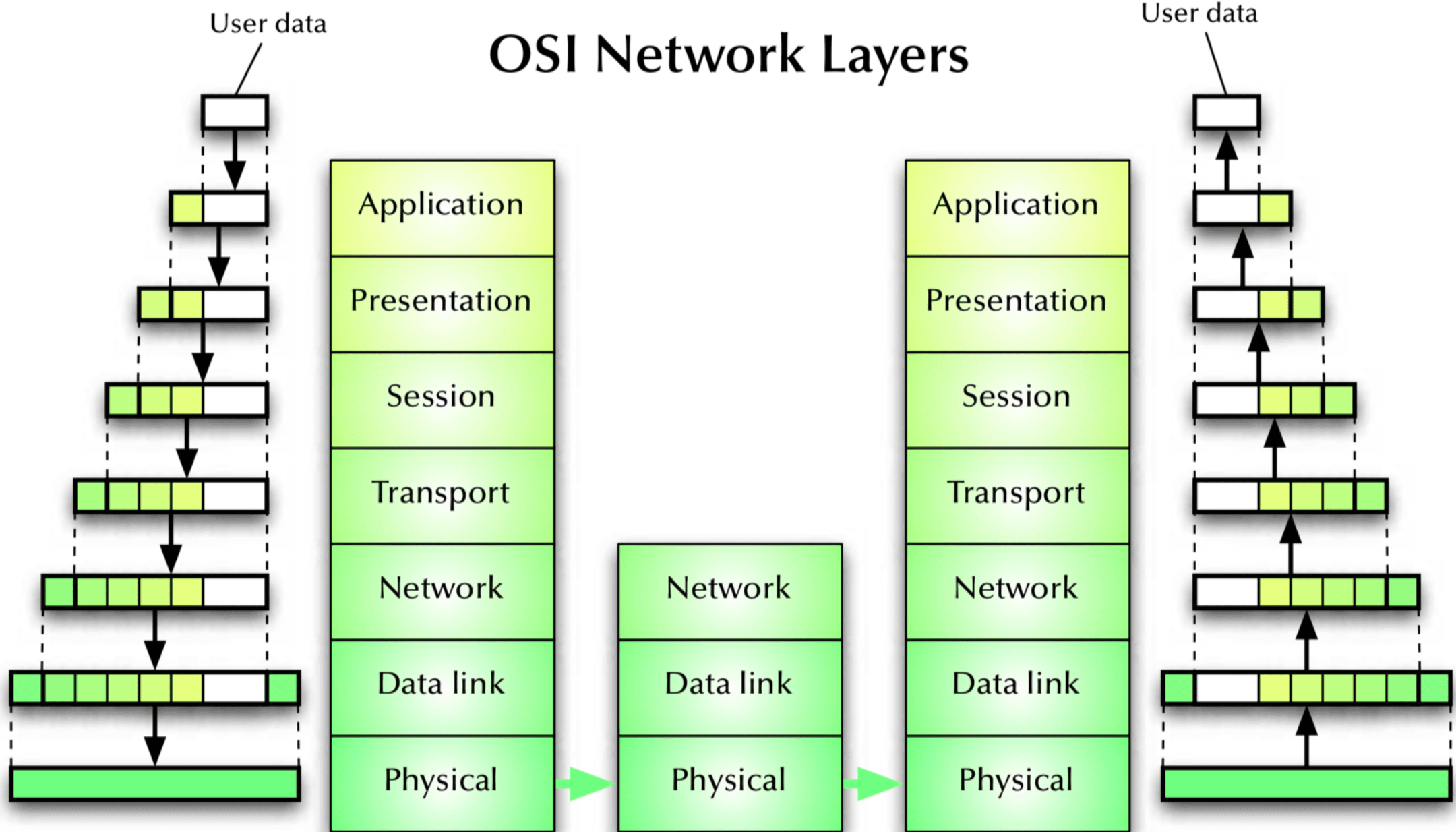


CC BY-SA Sparkfun/Mikegrusin

Synchronous:



OSI Network Layers



talk

How does the P2300 protocol fit into the OSI layer diagram?

Operating Systems

OS Main Topics (in short)

- Operating Systems: Concept
- OS Categories
- OS Architectures
- Processes - what are processes anyway?
- How do OSs handle processes?
- Scheduling

talk

Why do we need operating systems anyway?

What's an OS? ...two main roles

virtual machine

provides friendly & safe environment

memory management

hardware abstraction

process management

inter-process comms (IPC)

resource manager

coordinates access to resources

processors

memory

mass storage

communications channels

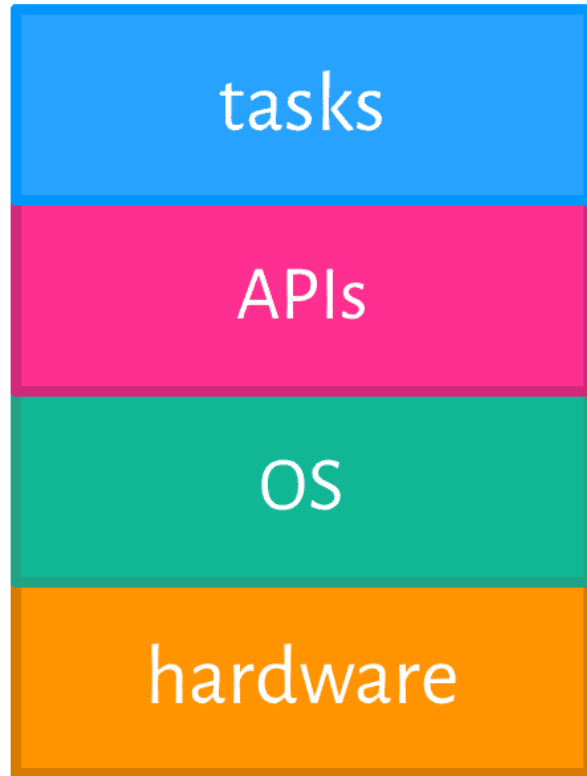
devices (timers, GPUs, DSPs, peripherals...)

Kernel: definition

the **kernel** is the program (functions, data structures in memory, etc.) which performs the *core* role(s) of the OS

access to the CPU, memory, peripherals all happens *through* the kernel through a *system call*

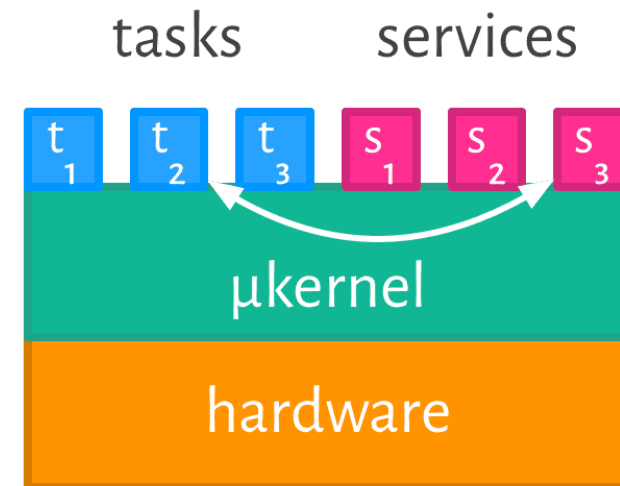
Monolithic OS - Modular - μ Kernels



monolithic OS



monolithic &



μ kernel OS

talk

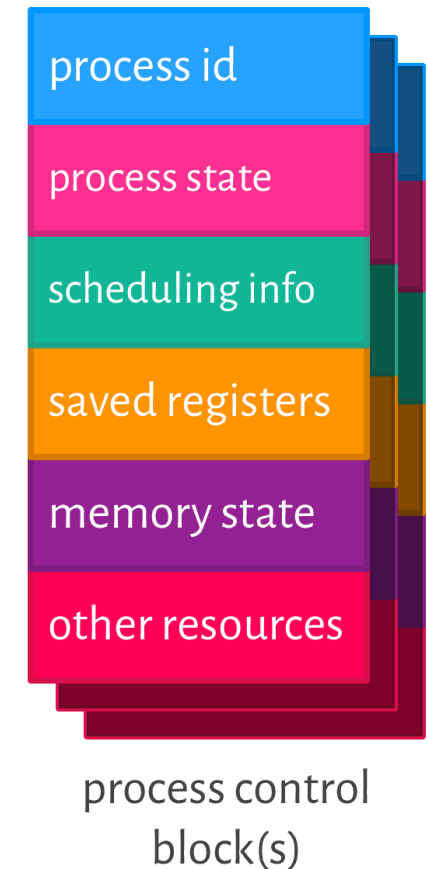
Why are kernels and (user) programs separate?

How are they different?

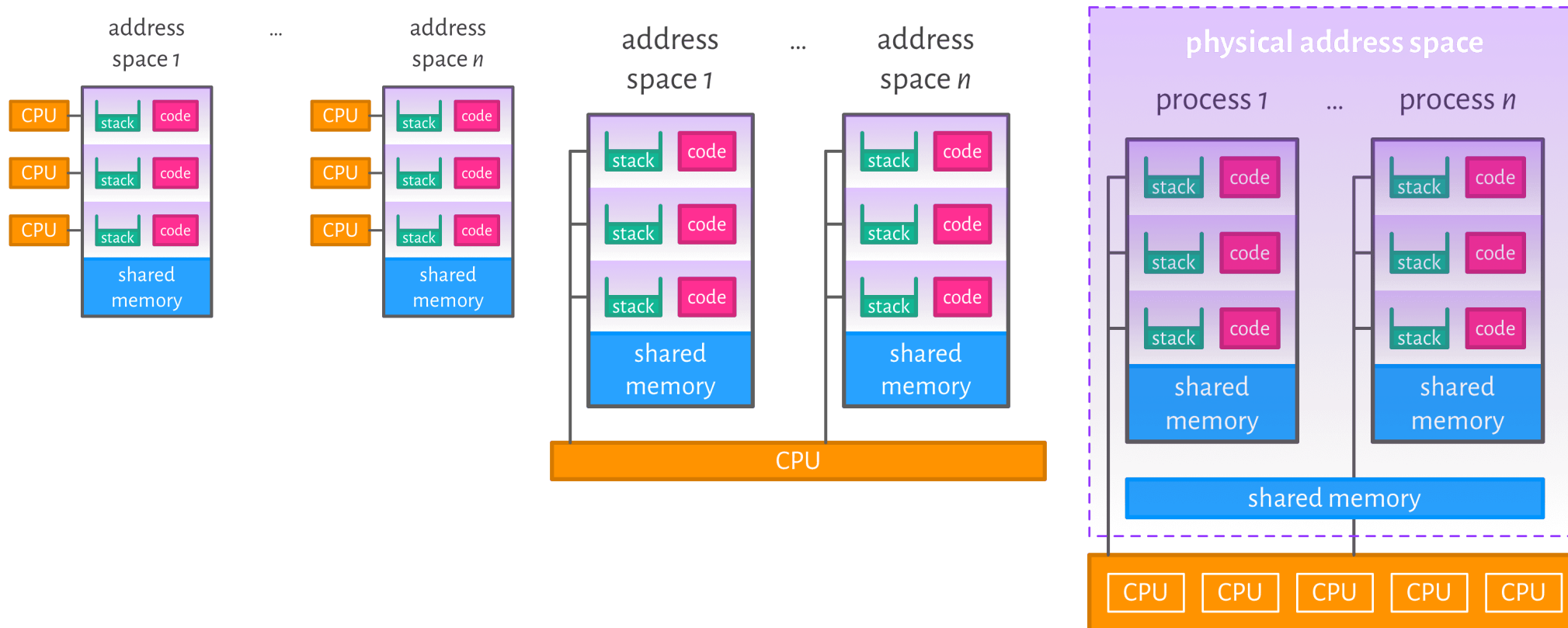
Aren't they both programs?

Process: definition

- a *running* program
- includes the code (instructions) for the program, and the current state/context:
- registers/flags
- memory (stack and heap)
- permissions/privileges
- other resources (e.g. global variables, open files & network connections, address space mappings)



Mapping processes to CPUs



Architecture

Architecture Main Topics (in short)

- History of computing architectures
- Harvard vs von Neumann architecture
- Pipelines
- Out-of-order execution
- Vector/SIMD instructions
- Hyper-threading
- Multi-core computing
- Virtual Memory
- Alternative architectures (Parallax Propeller)

A simple CPU

decoder/sequencer converts instructions into CPU control signals

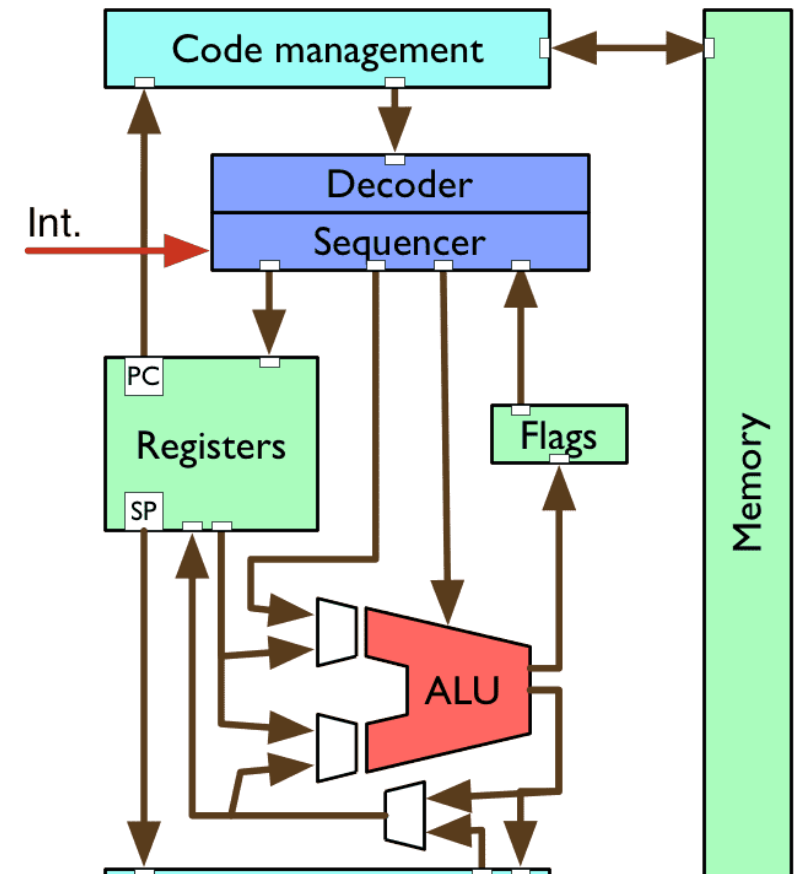
arithmetic logic unit (ALU) performs maths & logic operations

registers provide small, fast storage to the CPU

flags indicate the states of the latest calculations

code/data management for loading/storing, caching

memory



talk

What are the main components of a CPU? What do they do?

Are any components unnecessary?

Simple Pipeline

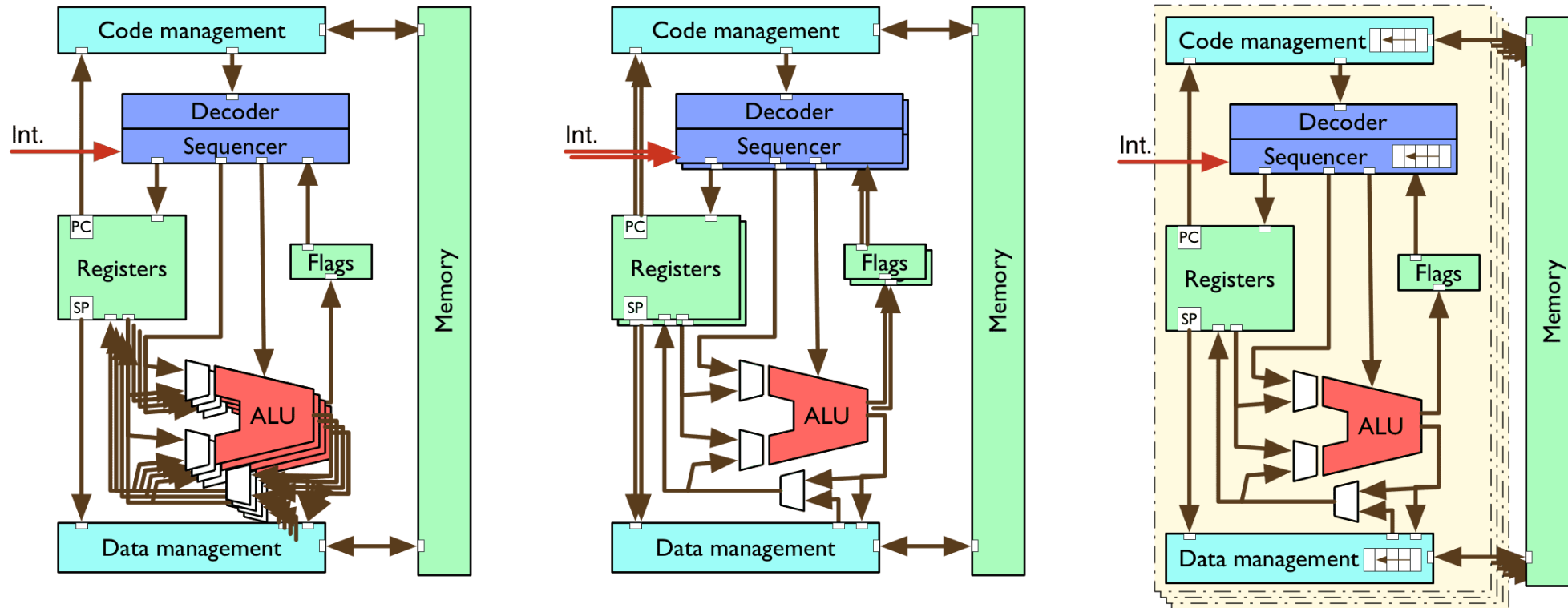
		Clock Cycles				
		1	2	3	4	5
Instructions	1	Fetch	Decode	Execute		
	2		Fetch	Decode	Execute	
	3			Fetch	Decode	Execute

talk

What is a CPU instruction pipeline and why would it be used?

Explain how it is possible to construct one.

Vector/SIMD vs hyper-threading vs multi-core

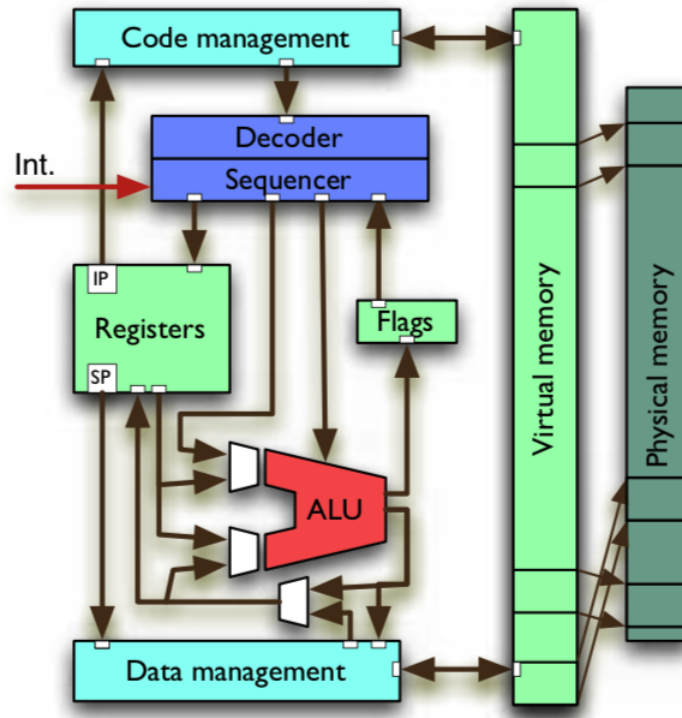


talk

what is the difference between Vector/SIMD, hyperthreading, and multi-core architectures?

What workloads benefit from each one?

Virtual Memory



Virtual memory

Translates logical memory addresses into physical memory addresses and provides memory protection features.

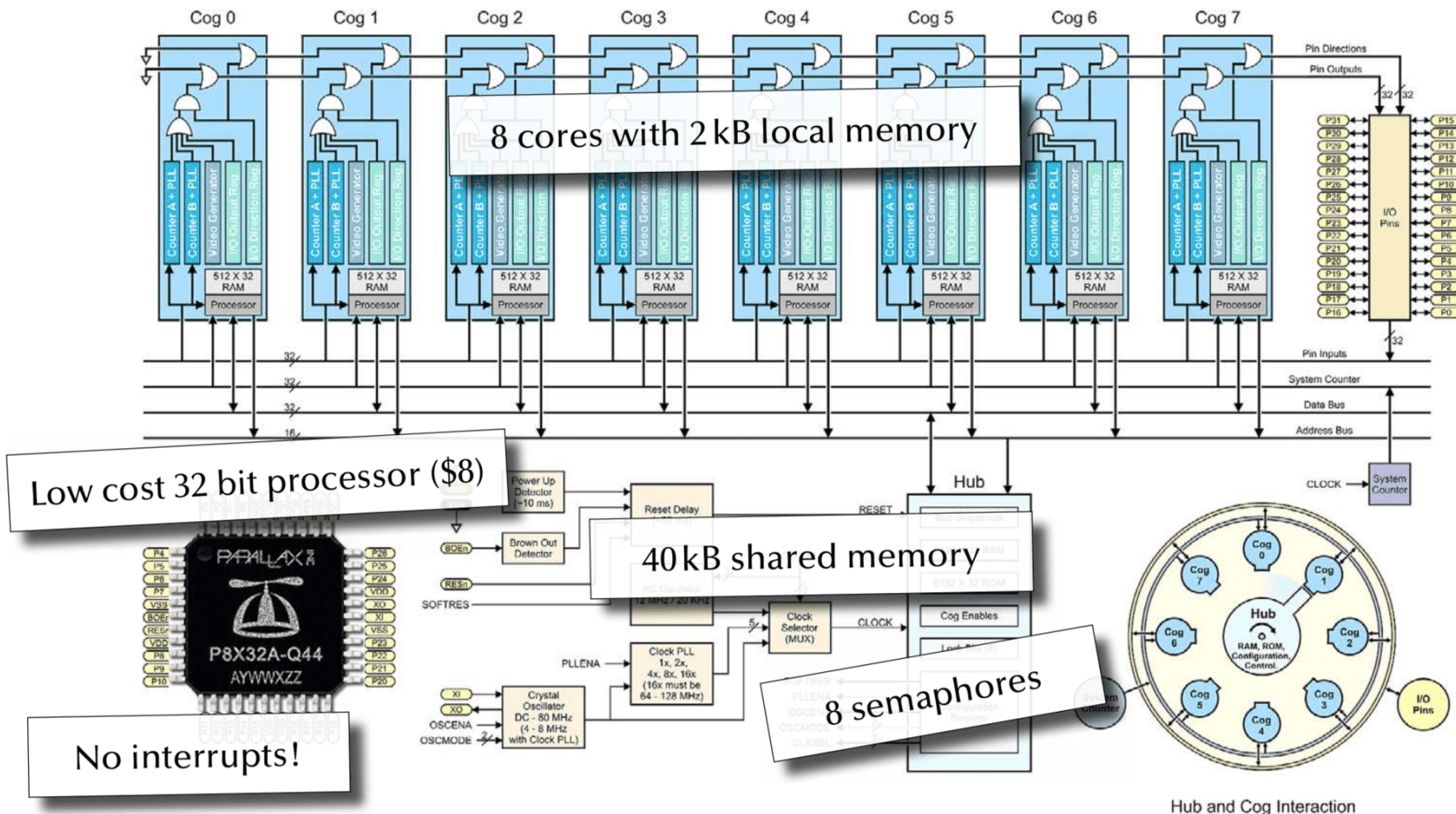
- Does not introduce concurrency by itself.
- ☞ Is still essential for concurrent programming as hardware memory protection guarantees memory integrity for individual processes / threads.

talk

Is virtual memory an architecture (hardware) topic or an OS (software) topic?

Explain reasons for both points of view.

Parallax Propeller



talk

What are the benefits of having many physical CPU cores? What problems could arise in practice?

Finally done. That was epic, thanks for
coming with me everybody!